

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn import tree
from sklearn.tree import DecisionTreeClassifier
from sklearn import metrics
import seaborn as sns
```

```
import kagglehub
# Download latest version
path = kagglehub.dataset_download("salihacur/diabetes")
print("Path to dataset files:", path)
```

Downloading to /root/.cache/kagglehub/datasets/salihacur/diabetes/2.archive...
 100%|██████████| 8.91k/8.91k [00:00<00:00, 5.15MB/s]Extracting files...
 Path to dataset files: /root/.cache/kagglehub/datasets/salihacur/diabetes/version_2

```
import os
print(os.listdir(path))
```

```
['diabetes.csv']
```

```
df = pd.read_csv(f"{path}/diabetes.csv")
print(df.shape)
df.head()
```

```
(768, 9)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPec
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
df.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

```
feature_cols = ["Pregnancies","Glucose","BloodPressure","Insulin","BMI","Age",
"DiabetesPedigreeFunction"]
x = df[feature_cols]
y = df.Outcome
print(x)
print(y)
```

	Pregnancies	Glucose	BloodPressure	Insulin	BMI	Age	\
0	6	148	72	0	33.6	50	
1	1	85	66	0	26.6	31	
2	8	183	64	0	23.3	32	
3	1	89	66	94	28.1	21	
4	0	137	40	168	43.1	33	
..	...	...	...	...	...	...	
763	10	101	76	180	32.9	63	
764	2	122	70	0	36.8	27	
765	5	121	72	112	26.2	30	
766	1	126	60	0	30.1	47	
767	1	93	70	0	30.4	23	

	DiabetesPedigreeFunction
0	0.627
1	0.351
2	0.672
3	0.167
4	2.288
..	...
763	0.171
764	0.340
765	0.245
766	0.349
767	0.315

```
[768 rows x 7 columns]
0      1
1      0
2      1
3      0
4      1
..
763    0
764    0
765    0
766    1
767    0
Name: Outcome, Length: 768, dtype: int64
```

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.25,random_stat
print(x_train.shape,y_train.shape,x_test.shape,y_test.shape)
```

```
(576, 7) (576,) (192, 7) (192,)
```

```
model = DecisionTreeClassifier(criterion='entropy',random_state =5)
model.fit(x_train,y_train)
```

▼ **DecisionTreeClassifier** ⓘ ?

DecisionTreeClassifier(criterion='entropy', random_state=5)

```
y_pred = model.predict(x_test)
print(y_pred)
```

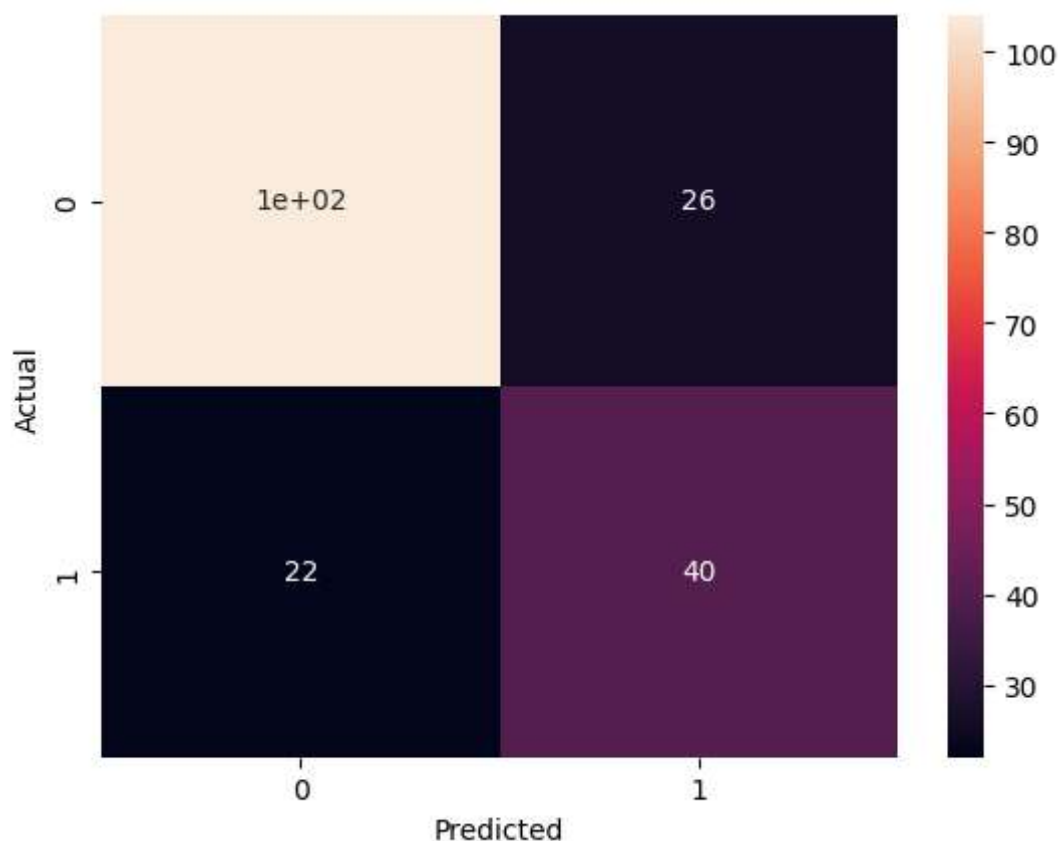
```
[1 0 0 1 0 0 1 1 0 1 1 1 0 0 0 1 1 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 1 1
 0 1 1 0 0 1 1 1 0 0 0 0 0 0 0 1 1 1 0 0 1 1 0 0 1 0 0 1 1 0 1 0 1 1 0 0 0 1
 1 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 0 0 1 0 0 0 0 1 0 0 0 0 0 1 0 1 0 0 0
 1 1 1 1 1 0 0 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 0 0 0
 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 1 1 1 0 1 0 0 0 0 1 1 0 0 0 0 0 0 1 0 1 1
 0 1 0 1 0 0 1]
```

```
conf_mat = metrics.confusion_matrix(y_test,y_pred)
print(conf_mat)
accuracy = metrics.accuracy_score(y_test,y_pred)
print(accuracy)
```

```
[[104  26]
 [ 22  40]]
0.75
```

```
conf_mat = pd.crosstab(y_test,y_pred,rownames=["Actual"],colnames=["Predicted"]
sns.heatmap(conf_mat,annot=True)
```

<Axes: xlabel='Predicted', ylabel='Actual'>



```
# Download latest version
path = kagglehub.dataset_download("dheemanthbhat/simple-weather-forecast")
print("Path to dataset files:", path)
```

```
Downloading to /root/.cache/kagglehub/datasets/dheemanthbhat/simple-weather-fore
100%|██████████| 322/322 [00:00<00:00, 664kB/s]Extracting files...
Path to dataset files: /root/.cache/kagglehub/datasets/dheemanthbhat/simple-weat
```

```
import os
print(os.listdir(path))
```

```
['weather_forecast.csv']
```

```
df = pd.read_csv(f"{path}/weather_forecast.csv")
df.head()
```

	Outlook	Temperature	Humidity	Windy	Play
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes

Next steps:

[Generate code with df](#)[New interactive sheet](#)

df.head()

	Outlook	Temperature	Humidity	Windy	Play
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df = pd.get_dummies(df, columns=['Outlook', 'Temperature', 'Humidity', 'Windy'])
df.head()
```

	Play	Outlook_Overcast	Outlook_Rain	Outlook_Sunny	Temperature_Cool	Temperature_Hot	Temperature_Mild	Humidity_High	Humidity_Normal	Windy_Weak	Windy_Strong
0	No	False	False	True	False	True	False	True	False	True	False
1	No	False	False	True	False	True	False	True	False	False	True
2	Yes	True	False	False	False	False	True	True	False	True	False
3	Yes	False	True	False	False	False	True	True	False	True	False
4	Yes	False	True	False	True	False	False	False	True	True	False

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
feature_cols = ["Outlook_Overcast", "Outlook_Rain", "Outlook_Sunny", "Temperature_Cool", "Temperature_Hot", "Temperature_Mild", "Humidity_High", "Humidity_Normal", "Windy_Weak", "Windy_Strong"]
x = df[feature_cols]
y = df.Play
```

```
print(x)
print(y)
```

	Outlook_Overcast	Outlook_Rain	Outlook_Sunny	Temperature_Cool	\
0	False	False	True	False	
1	False	False	True	False	
2	True	False	False	False	
3	False	True	False	False	
4	False	True	False	True	
5	False	True	False	True	
6	True	False	False	True	
7	False	False	True	False	
8	False	False	True	True	
9	False	True	False	False	
10	False	False	True	False	
11	True	False	False	False	
12	True	False	False	False	
13	False	True	False	False	

	Temperature_Hot	Temperature_Mild	Humidity_High	Humidity_Normal	\
0	True	False	True	False	
1	True	False	True	False	
2	True	False	True	False	
3	False	True	True	False	
4	False	False	False	True	
5	False	False	False	True	
6	False	False	False	True	
7	False	True	True	False	
8	False	False	False	True	
9	False	True	False	True	
10	False	True	False	True	
11	False	True	True	False	
12	True	False	False	True	
13	False	True	True	False	

	Windy_Strong	Windy_Weak
0	False	True
1	True	False
2	False	True
3	False	True
4	False	True
5	True	False
6	True	False
7	False	True
8	False	True
9	False	True
10	True	False
11	True	False
12	False	True
13	True	False

0	No
1	No
2	Yes
3	Yes
4	Yes
5	No

```
6    Yes
7    No
8    Yes
9    Yes
10   Yes
```

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.25,random_stat
print(x_train.shape,y_train.shape,x_test.shape,y_test.shape)
```

```
(10, 10) (10,) (4, 10) (4,)
```

```
model = DecisionTreeClassifier(criterion='entropy',random_state =5)
model.fit(x_train,y_train)
```

▼ DecisionTreeClassifier ⓘ ?

```
DecisionTreeClassifier(criterion='entropy', random_state=5)
```

```
y_pred = model.predict(x_test)
print(y_pred)
```

```
['No' 'Yes' 'No' 'Yes']
```

```
conf_mat = metrics.confusion_matrix(y_test,y_pred)
print(conf_mat)
accuracy = metrics.accuracy_score(y_test,y_pred)
print(accuracy)
```

```
[[0 0]
 [2 2]]
0.5
```